

1. Описание условия задания

Цель работы:

- реализовать операции работы со стеком, который представлен в виде статического массива и в виде односвязного линейного списка;
- оценить преимущества и недостатки каждой реализации;
- получить представление о механизмах выделения и освобождения памяти при работе со стеком.

Задача:

- Создать программу работы со стеком, выполняющую операции добавления, удаления элементов и вывод текущего состояния стека.

Реализовать стек:

- а) статическим массивом;
- б) односвязным списком.

Задача по варианту:

- Ввести целые числа в 2 стека. Используя третий стек отсортировать все введенные данные.

2. Техническое задание

Внешняя спецификация программы:

Исходные данные:

- Элементы стека в виде целых чисел, вводимых пользователем.
- Выбор операции через консольное меню.

Выходные данные:

- Текущее состояние стека.
- Адреса элементов (для списка).
- История удаленных элементов (для списка).
- Результаты сортировки двух стеков с помощью третьего.
- Сравнение времени выполнения и объема памяти для двух реализаций.

Ограничения:

- Максимальный размер стека: 10 000 элементов.

Способ обращения: консольный ввод/вывод.

Команда запуска программы: ./app.exe. Дальнейшее взаимодействие осуществляется с помощью меню.

Аварийные ситуации:

- “ERROR_MEM” – Ошибка выделения памяти.
- “ERROR_OVERFLOW” – Попытка добавить элемент в полный стек.
- “ERROR_DELETING” – Попытка удалить элемент из пустого стека.
- “ERROR_INPUT” – Попытка ввести некорректное значение.

Возможные действия, реализуемой программой:

0. Выход.
1. Эмуляция стека.
 - a. Назад.
 - b. Стек в виде статического массива.
 - i. Назад.
 - ii. Добавление элемента.
 - iii. Удалить элемент.
 - iv. Отсортировать два стека с помощью третьего.
 - v. Вывод содержимого стека.
 - c. Стек в виде односвязного списка.
 - i. Назад.
 - ii. Добавить элемент.
 - iii. Удалить элемент.
 - iv. Отсортировать два стека с помощью третьего.
 - v. Вывод содержимого стека.
 - vi. Вывод содержимого стека с адресами.
 - vii. Вывод освобождённых элементов.
2. Сравнение производительности.

3. Внутренние структуры данных

В рамках выполнения данной лабораторной работы мною были реализованы следующие структуры данных:

c
<pre>typedef struct { int data[MAX_SIZE]; // MAX_SIZE = 10 000 int top; } ArrayStack;</pre>

Данная структура данных описывает стек в виде статического массива:

- **data** – массив элементов стека.
- **top** – порядковый номер вершины стека.

c
<pre>typedef struct { Node* top; int size; } ListStack;</pre>

Данная структура данных описывает стек в виде односвязного списка:

- **top** – указатель на вершину стека, то есть на голову списка.
- **size** – количество элементов стека.

c
<pre>typedef struct Node { int data; struct Node* next; } Node;</pre>

Данная структура данных описывает узел односвязного списка:

- **data** – значение элемента списка, его информационная часть.
- **next** – указатель на следующий узел списка, его ссылочная часть.

4. Описание Алгоритмов

Основной алгоритм программы:

1. Вывести меню;
2. Считать выбор пользователя и проверить его корректность;
3. Выполнить соответствующее действие;
4. Либо снова вывести меню с ожиданием выбора, либо завершить программу с сообщением об ошибке.

Алгоритм добавления элемента:

1. Проверить наполненность стека.
2. Для списка выделить память под новый узел.
3. Увеличить счётчик количества элементов и для списка связать новый элемент с предыдущим.

Алгоритм удаления элемента:

1. Проверить наличие элементов в стеке.
2. Для списка сохранить адрес удаляемого элемента.
3. Прочитать значение удаляемого элемента.
4. Освободить узел для списка и уменьшить счётчик количества элементов.

Алгоритм сортировки двух стеков с помощью третьего:

1. Перенести элементы из стеков во временный стек.
2. Сортировать элементы стека с помощью результирующего стека.
3. Вывести отсортированный стек.

Алгоритм сравнения производительности:

1. Рассчитать различные размеры стеков.
2. Сгенерировать стек в виде массива и в виде списка.
3. Измерить время сортировки.
4. Рассчитать занимаемую память.
5. Вывести результат.

Реализуемые функции:

- // Функция эмуляции стека*
1. void emulate_stack(void)
- // Функция сравнения производительности*
2. void compare_stacks(void)
- // Функция ввода числа*
3. int get_integer_input(const char* prompt, int min, int max)
- // Функция очистки буфера*
4. void clear_input_buffer(void)
- // Функция вывода начального меню*
5. void print_main_menu(void)
- // Функция выбора способа использования стека*
6. void print_stack_type_menu(void)
- // Функция вывода меню для взаимодействия со стеком*
7. void print_operation_menu(int stack_type)
- // Функция взаимодействий со стеком в виде списка*
8. void list_stack_operations(ListStack* stack)
- // Функция инициализации стека в виде списка*
9. void list_init(ListStack* stack)
- // Функция проверки стека на отсутствие элементов*
10. int list_is_empty(ListStack* stack)
- // Функция проверки стека на переполнение*
11. int list_is_full(ListStack* stack)
- // Функция для добавление нового элемента в стек*
12. int list_push(ListStack* stack, int value)
- // Функция для удаления последнего элемента из стека*
13. int list_pop(ListStack* stack, int* value)
- // Функция вывода стека*
14. void list_display(ListStack* stack)

```
// Функция вывода стека с адресами
15. void list_display_with_addresses(ListStack* stack)

// Функция освобождения стека в виде списка
16. void list_free(ListStack* stack)

// Функция сортировки двух стеков с помощью третьего
17. void sort_with_list_stack(void)

// Функции добавления свободных адресов
18. void add_free_address(void* addr)

// Функция вывода свободных адресов
19. void display_free_addresses(void)

// Функция очистки свободных адресов
20. void clear_free_addresses(void)

// Функция взаимодействий со стеком в виде массива
21. void array_stack_operations(ArrayStack* stack)

// Функция инициализации стека в виде массива
22. void array_init(ArrayStack* stack)

// Функция проверки стека на отсутствие элементов
23. int array_is_empty(ArrayStack* stack)

// Функция проверки стека на переполнение
24. int array_is_full(ArrayStack* stack)

// Функция для добавление нового элемента в стек
25. int array_push(ArrayStack* stack, int value)

// Функция для удаления последнего элемента из стека
26. int array_pop(ArrayStack* stack, int* value)

// Функция вывода стека
27. void array_display(ArrayStack* stack)

// Функция сортировки двух стеков с помощью третьего
28. void sort_with_array_stack(void)
```

5. Тесты

Позитивные тесты:
1. Добавление элемента в стек в виде массива. 2. Добавление элемента в стек в виде списка. 3. Удаление из непустого стека в виде массива. 4. Удаление из непустого стека в виде списка. 5. Вывод стека в виде массива. 6. Вывод стека в виде списка. 7. Вывод стека в виде списка с адресами элементов. 8. Вывод освобождённых элементов стека в виде списка. 9. Сортировка двух непустых стеков с помощью третьего в виде массива. 10. Сортировка двух непустых стеков с помощью третьего в виде массива. 11. Сравнение эффективности сортировки стеков.

Негативные тесты:
1. Удаление из пустого стека. 2. Добавление элемента в полный стек. 3. Ввод невалидного значения.

6. Оценка эффективности

Количество запусков для каждого размера – 30 раз.

Размер стека	Время массива, с	Время списка, с	Отношение времени (Массив/Список)	Память массива, байт	Память списка, байт	Отношение памяти (Массив/Список)
3	0.000001	0.000001	0.820513	160000	192	833.333
5	0.000001	0.000002	0.711864	160000	320	500.000
10	0.000003	0.000007	0.451282	160000	640	250.000
20	0.000007	0.000017	0.379048	160000	1280	125.000
50	0.000028	0.000089	0.320030	160000	3200	50.000
100	0.000065	0.000228	0.286715	160000	6400	25.000
300	0.000277	0.001101	0.251445	160000	19200	8.333
500	0.000680	0.003055	0.222527	160000	32000	5.000
1000	0.002732	0.013624	0.200560	160000	64000	2.500
2500	0.016629	0.090837	0.183068	160000	160000	1.000
3000	0.023843	0.161298	0.147817	160000	192000	0.833
5000	0.065625	0.420950	0.155898	160000	320000	0.500
10000	0.261393	1.558363	0.167736	160000	640000	0.250

Наблюдение результатов по времени:

- Реализация на массиве демонстрирует стабильное преимущество по времени выполнения.
- Отношение времени (массив/список) составляет 0.15-0.82, что свидетельствует о 18-85% экономии времени при использовании массива.
- С ростом размера стека преимущество массива сохраняется.

Критические точки для памяти:

- При $n < 2500$: список экономичнее (отношение памяти 1.0-833.3)
- При $n = 2500$: равные затраты памяти
- При $n > 2500$: массив экономичнее

7. Выводы

Итак, можно сделать следующие выводы:

1. По временной эффективности:

Массив значительно превосходит список - в среднем операции со стеком на массиве выполняются в 5-8 раз быстрее. Это объясняется:

- Отсутствием затрат на динамическое выделение/освобождение памяти.
- Локальностью данных.
- Минимальными накладными расходами на операции push/pop.

2. По емкостной эффективности:

Список экономичнее при $n < 2500$ - использует память пропорционально фактическому количеству элементов, тогда как массив всегда резервирует максимальный объем (160 000 байт).

Для задач работы со стеком, где размер известен и не превышает разумных пределов, реализация на массиве предпочтительнее благодаря значительному преимуществу в скорости выполнения операций. Реализация на списке целесообразна в специфических сценариях с требованиями к динамическому изменению размера или строгой экономии памяти.

При использовании списка для стека возникает фрагментация памяти, то есть образуются несмежные свободные участки памяти между занятыми блоками. Это происходит из-за частого динамического выделения и освобождения памяти под узлы списка. Однако для стека данная проблема не является критичной, поскольку менеджер памяти эффективно перераспределяет освобожденные блоки благодаря предсказуемому LIFO-характеру операций.

8. Ответы на контрольные вопросы

1. Что такое стек?

Стек — это абстрактный тип данных (АТД), представляющий собой последовательный список с переменной длиной, в котором включение и исключение элементов происходит только с одной стороны — вершины стека.

- Принцип работы: LIFO (Last In — First Out) — «последним пришел, первым ушел».
- Доступен только верхний элемент (вершина стека).
- Основные операции: push (добавление), pop (удаление).
- Классическая реализация не позволяет просматривать содержимое без извлечения элементов.

2. Каким образом и сколько памяти выделяется под хранение стека при различной его реализации?

Реализация массивом:

- Выделение памяти: единым непрерывным блоком фиксированного размера.
- Объем памяти: $MAX_SIZE \times \text{sizeof}(\text{тип_элемента})$.
- Особенности: память выделяется сразу вся, даже если стек пуст.

Реализация списком:

- Выделение памяти: динамически под каждый новый элемент.
- Объем памяти: $n \times \text{sizeof}(\text{Node})$, где Node содержит данные и указатель.
- Особенности: память выделяется только для фактически хранимых элементов.

3. Каким образом освобождается память при удалении элемента стека при различной реализации стека?

Реализация массивом:

- Процесс: уменьшение указателя вершины ($top--$).
- Освобождение памяти: фактически не происходит — память остается зарезервированной.
- Преимущество: минимальные накладные расходы.
- Недостаток: неэффективное использование памяти при неполном заполнении.

Реализация списком:

- Процесс:
 - o Сохранение ссылки на удаляемый элемент.
 - o Перемещение указателя вершины к следующему элементу.
 - o Явное освобождение памяти через free().
 - o Добавление адреса в список освобожденной памяти для отслеживания.
- Преимущество: оптимальное использование памяти.
- Недостаток: дополнительные затраты на управление памятью.

4. Что происходит с элементами стека при его просмотре?

При классическом подходе:

- Просмотр невозможен без извлечения элементов.
- Для доступа к элементам необходимо последовательно извлекать их через pop().

В реализованной программе:

- Массив: элементы выводятся от вершины к основанию без изменения стека.
- Список: возможен просмотр с отображением адресов элементов.

5. Каким образом эффективнее реализовывать стек? От чего это зависит?

Реализация массивом эффективнее, когда:

- Известен максимальный размер стека.
- Требуется высокая производительность.
- Важна локальность данных для эффективного использования кэша процессора.

Реализация списком эффективнее, когда:

- Размер стека непредсказуем или сильно варьируется.
- Критична экономия памяти.
- Требуется гибкость структуры данных.
- Необходимо динамическое изменение размера в широких пределах.